

Namaa — End-to-End Implementation Plan & Architecture Blueprint

A Privacy-First AI Operations Copilot for SMEs

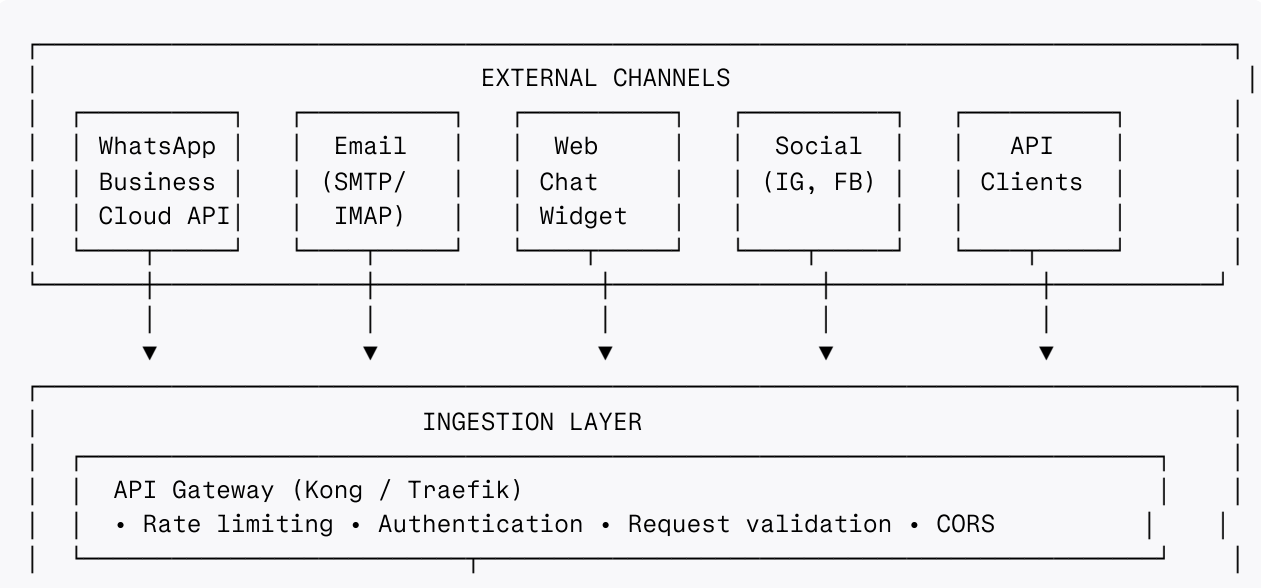
Mission: Make every growing business as operationally capable as a large enterprise — securely, bilingually, and without complexity.

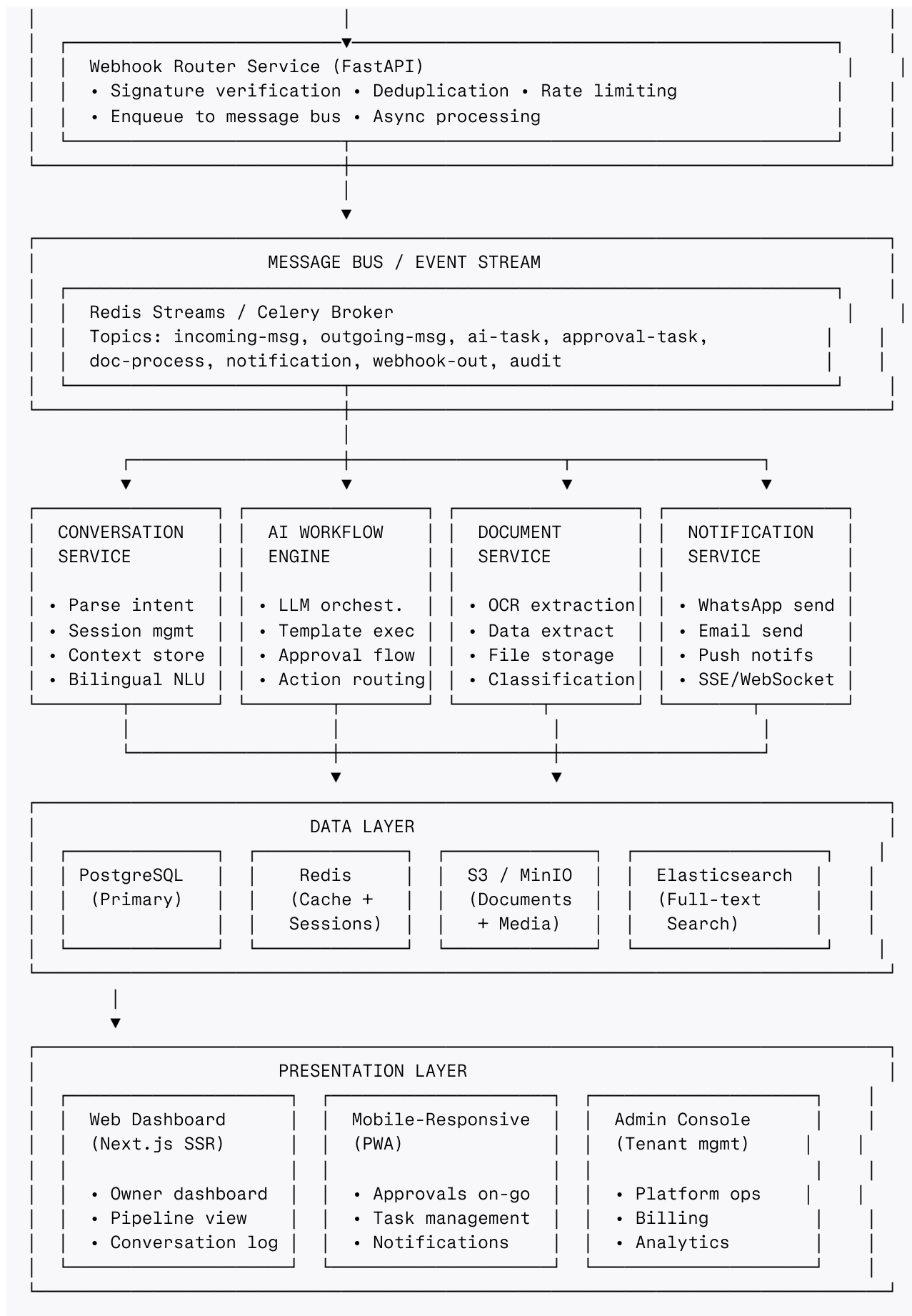
Table of Contents

- 1. [System Topology](#)
- 2. [Tech Stack Selection](#)
- 3. [Backend API Design](#)
- 4. [Database Schema](#)
- 5. [Webhook Integration Flow](#)
- 6. [AI & Workflow Engine Design](#)
- 7. [Security & Compliance Architecture](#)
- 8. [Phased Deployment Roadmap](#)
- 9. [Milestones & Deliverables](#)
- 10. [Infrastructure & DevOps](#)

1. System Topology

High-Level Architecture Diagram (ASCII)

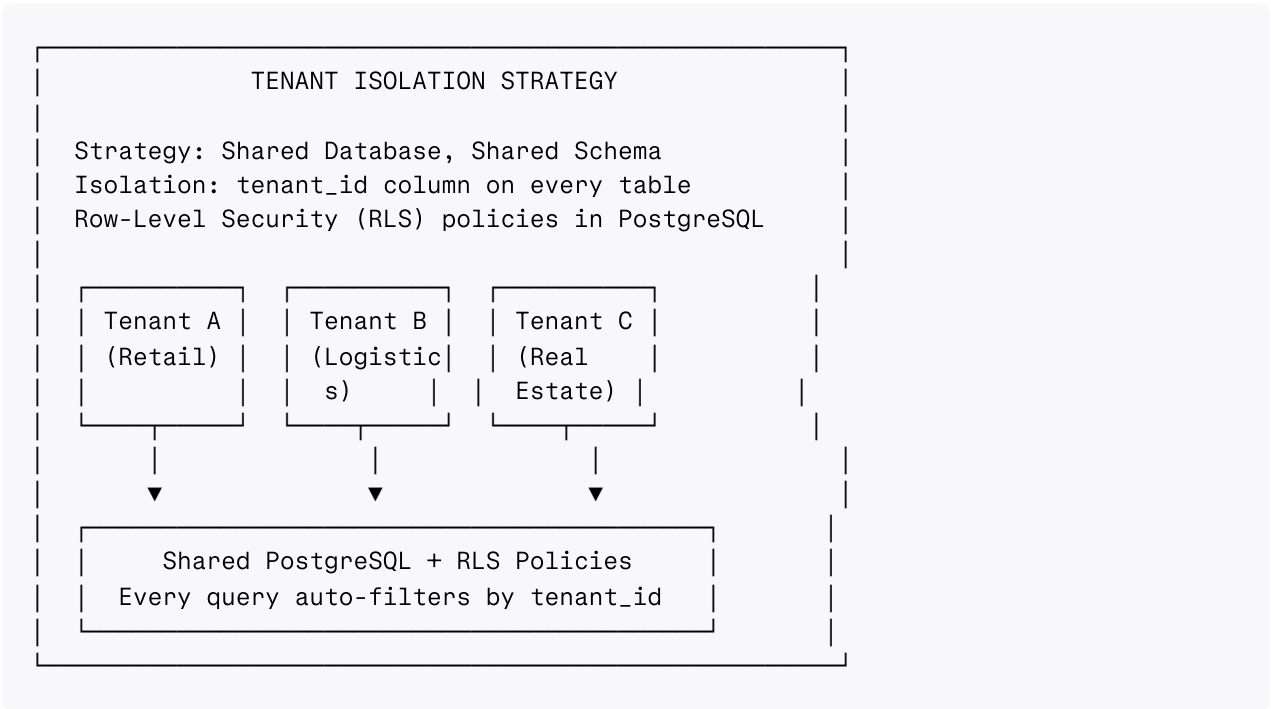




Topology Components Explained

Layer	Purpose	Key Services
External Channels	Where messages originate — WhatsApp, email, web chat, social media	WhatsApp Business Cloud API, SMTP/IMAP, WebSocket widget
Ingestion Layer	Receives, validates, and routes all incoming messages	API Gateway, Webhook Router Service
Message Bus	Decouples producers from consumers; enables async processing	Redis Streams + Celery workers
Core Services	Business logic: conversation handling, AI orchestration, document processing, notifications	Conversation Service, AI Workflow Engine, Document Service, Notification Service
Data Layer	Persistent storage, caching, file storage, and search	PostgreSQL, Redis, S3/MinIO, Elasticsearch
Presentation Layer	User-facing interfaces for owners, staff, and admins	Next.js dashboard, PWA, admin console

Multi-Tenancy Model



Rationale: Shared-database multi-tenancy is the right choice for a startup. It keeps costs low, simplifies operations, and scales to thousands of tenants before needing to shard. Row-Level Security in PostgreSQL provides strong isolation guarantees without the operational overhead of separate databases per tenant.

2. Tech Stack Selection

Backend

Component	Technology	Rationale
Primary Language	Python 3.12+	Best AI/ML ecosystem; FastAPI for async; Celery for background jobs; LangChain/LlamaIndex for LLM orchestration
Web Framework	FastAPI	Async-native, OpenAPI auto-generation, type validation via Pydantic, WebSocket support, high performance
Background Jobs	Celery + Redis	Battle-tested task queue; supports retries, scheduling, priority queues, task chaining
ORM	SQLAlchemy 2.0 (async)	Mature, async support, RLS compatibility, Alembic migrations
Database Migrations	Alembic	Version-controlled schema evolution, rollback support
Validation	Pydantic v2	Native to FastAPI; schema validation for all inputs and outputs

Frontend

Component	Technology	Rationale
Framework	Next.js 14+ (App Router)	SSR for dashboard performance, SEO for public pages, API routes for BFF pattern
UI Library	Tailwind CSS + shadcn/ui	Rapid, consistent, accessible components; fully customizable
State Management	Zustand + TanStack Query	Lightweight state; server-state caching, optimistic updates
Real-time	Server-Sent Events (SSE) + WebSocket	SSE for dashboard updates; WebSocket for live chat widget
Internationalization	next-intl	Arabic (RTL) + English (LTR) with proper direction switching
Charts	Recharts / Tremor	Owner dashboard metrics, ROI visualization

Data Layer

Component	Technology	Rationale
Primary Database	PostgreSQL 16	RLS for multi-tenancy, JSONB for flexible data, full-text search (Arabic + English), partitioning for scale
Cache / Session Store	Redis 7	Conversation context, rate limiting, Celery broker, pub/sub for real-time
Object Storage	MinIO (self-hosted) or AWS S3	Document storage (invoices, contracts, forms); presigned URLs for secure access
Search Engine	Elasticsearch 8 (optional, Phase 2)	Advanced search across conversations, documents, customer history — Arabic analyzers available

Component	Technology	Rationale
Time-Series	TimescaleDB (PostgreSQL extension)	ROI metrics, usage analytics, performance dashboards

AI / LLM Layer

Component	Technology	Rationale
LLM Provider (Primary)	OpenAI GPT-4o / GPT-4o-mini	Strong multilingual (Arabic + English), function calling, structured output
LLM Provider (Fallback)	Anthropic Claude 3.5 Sonnet	Alternative for complex reasoning; fallback for rate limits
Embedding Model	text-embedding-3-small (OpenAI)	For semantic search, customer history retrieval
Vector Store	pgvector (PostgreSQL extension)	No separate infrastructure; in-DB vector search for RAG
OCR / Document Intelligence	Google Cloud Document AI (primary) + Tesseract (fallback)	Structured extraction from invoices, contracts, forms; Tesseract for cost-sensitive tasks
Workflow Orchestration	Custom lightweight engine (Celery-based)	Predictable, debuggable; avoid heavy frameworks initially

Infrastructure & DevOps

Component	Technology	Rationale
Container Runtime	Docker	Standard packaging
Orchestration	Docker Compose (dev) → Kubernetes (prod)	Start simple; migrate to K8s at scale
CI/CD	GitHub Actions	Test, build, deploy pipeline; environment promotion
Reverse Proxy / TLS	Traefik or Caddy	Auto TLS, simple config, load balancing
Monitoring	Prometheus + Grafana	Metrics, dashboards, alerting
Logging	Loki + Promtail (or ELK)	Structured JSON logging, centralized search
Tracing	OpenTelemetry + Jaeger	Distributed tracing across services
Secrets Management	HashiCorp Vault or AWS Secrets Manager	API keys, DB credentials, encryption keys

Communication Integrations

Integration	Technology	Rationale
WhatsApp	WhatsApp Business Cloud API (Meta)	Official, cloud-hosted, webhook-based, supports Arabic templates
Email	SMTP (outbound) + IMAP polling (inbound)	Universal; later upgrade to Microsoft Graph / Gmail API for richer integration
SMS	Twilio or Unifonic	Unifonic for GCC-local delivery; Twilio for global

Integration	Technology	Rationale
Push Notifications	Firebase Cloud Messaging (FCM)	Free, cross-platform for PWA/mobile

3. Backend API Design

API Principles

- **RESTful** for CRUD operations and resource management
- **WebSocket** for real-time chat widget and live dashboard updates
- **Webhook endpoints** for inbound message ingestion (WhatsApp, email)
- **Versioned**: All endpoints under `/api/v1/`
- **Auth**: JWT (access + refresh), scoped per tenant and role
- **Rate limiting**: Per-tenant and per-endpoint limits in API Gateway
- **Pagination**: Cursor-based for large collections
- **Errors**: Consistent JSON error envelope with error codes

Authentication & Authorization

```

POST    /api/v1/auth/register      # Tenant registration
POST    /api/v1/auth/login        # Login (email/password)
POST    /api/v1/auth/refresh      # Refresh access token
POST    /api/v1/auth/logout       # Revoke refresh token
POST    /api/v1/auth/otp/request  # Request OTP (phone/email)
POST    /api/v1/auth/otp/verify   # Verify OTP
GET     /api/v1/auth/me           # Current user profile

```

Conversation Management

```

GET     /api/v1/conversations      # List conversations (paginated)
GET     /api/v1/conversations/{conversation_id} # Get conversation detail
GET     /api/v1/conversations/{conversation_id}/messages # Get messages
POST    /api/v1/conversations/{conversation_id}/messages # Send message (manual)
PATCH  /api/v1/conversations/{conversation_id} # Update (assign, close, etc.)
POST    /api/v1/conversations/{conversation_id}/approve # Approve AI-drafted message
POST    /api/v1/conversations/{conversation_id}/reject # Reject AI-drafted message
GET     /api/v1/conversations/unassigned # Unassigned conversations
GET     /api/v1/conversations/pending-approval # Messages awaiting human review

```

Customer & Lead Management

GET	/api/v1/customers	# List customers
POST	/api/v1/customers	# Create customer
GET	/api/v1/customers/{customer_id}	# Get customer detail
PATCH	/api/v1/customers/{customer_id}	# Update customer
GET	/api/v1/customers/{customer_id}/history	# Conversation + interaction history
DELETE	/api/v1/customers/{customer_id}	# Soft delete (GDPR/PDPL)
GET	/api/v1/leads	# List leads (pipeline)
POST	/api/v1/leads	# Create lead
PATCH	/api/v1/leads/{lead_id}	# Update lead status
POST	/api/v1/leads/{lead_id}/convert	# Convert to customer
GET	/api/v1/leads/stats	# Pipeline metrics

Quotation & Document Management

GET	/api/v1/quotations	# List quotations
POST	/api/v1/quotations	# Create quotation (manual or AI)
GET	/api/v1/quotations/{quotation_id}	# Get quotation detail
PATCH	/api/v1/quotations/{quotation_id}	# Update quotation
POST	/api/v1/quotations/{quotation_id}/send	# Send quotation (WhatsApp/email)
POST	/api/v1/quotations/{quotation_id}/approve	# Approve quotation
POST	/api/v1/quotations/{quotation_id}/reject	# Reject quotation
GET	/api/v1/documents	# List documents
POST	/api/v1/documents/upload	# Upload document
GET	/api/v1/documents/{document_id}	# Get document detail
GET	/api/v1/documents/{document_id}/extracted	# Get extracted data
POST	/api/v1/documents/{document_id}/re-extract	# Re-run extraction

Workflow & Task Management

GET	/api/v1/tasks	# List tasks (action items)
POST	/api/v1/tasks	# Create task
PATCH	/api/v1/tasks/{task_id}	# Update task
POST	/api/v1/tasks/{task_id}/complete	# Mark complete
GET	/api/v1/tasks/today	# Today's action items for owner
GET	/api/v1/workflows	# List workflow templates
POST	/api/v1/workflows	# Create workflow template
GET	/api/v1/workflows/{workflow_id}	# Get workflow detail
POST	/api/v1/workflows/{workflow_id}/execute	# Execute workflow
GET	/api/v1/workflows/{workflow_id}/runs	# List workflow executions

Approval & Governance

GET	/api/v1/approvals	# List pending approvals
POST	/api/v1/approvals/{approval_id}/approve	# Approve action
POST	/api/v1/approvals/{approval_id}/reject	# Reject action
GET	/api/v1/approvals/history	# Approval history
GET	/api/v1/audit-log	# Audit trail (filterable)
GET	/api/v1/audit-log/export	# Export audit trail (CSV/PDF)

Dashboard & Analytics

GET	/api/v1/dashboard/today	# "What needs attention today"
GET	/api/v1/dashboard/metrics	# Key metrics (leads, quotes, etc)
GET	/api/v1/dashboard/roi	# ROI metrics (hours saved, leads generated)
GET	/api/v1/dashboard/conversation-stats	# Conversation volume, response times
GET	/api/v1/dashboard/staff-performance	# Staff activity metrics

Integration Management

GET	/api/v1/integrations	# List connected integrations
POST	/api/v1/integrations/whatsapp/connect	# Connect WhatsApp Business
POST	/api/v1/integrations/email/connect	# Connect email (SMTP/IMAP)
POST	/api/v1/integrations/calendar/connect	# Connect calendar
DELETE	/api/v1/integrations/{integration_id}	# Disconnect integration
GET	/api/v1/integrations/webhooks	# List webhook endpoints
POST	/api/v1/integrations/webhooks	# Register outbound webhook

Settings & Configuration

GET	/api/v1/settings	# Tenant settings
PATCH	/api/v1/settings	# Update settings
GET	/api/v1/settings/roles	# List roles
POST	/api/v1/settings/roles	# Create custom role
GET	/api/v1/settings/users	# List users
POST	/api/v1/settings/users	# Invite user
PATCH	/api/v1/settings/users/{user_id}	# Update user role
GET	/api/v1/settings/retention	# Data retention policies
PATCH	/api/v1/settings/retention	# Update retention policies
GET	/api/v1/settings/consent	# Consent records
PATCH	/api/v1/settings/consent	# Update consent settings

Product Catalog

GET	/api/v1/catalog/products	# List products/services
POST	/api/v1/catalog/products	# Create product
PATCH	/api/v1/catalog/products/{product_id}	# Update product

```
DELETE /api/v1/catalog/products/{product_id} # Delete product
GET    /api/v1/catalog/categories           # List categories
```

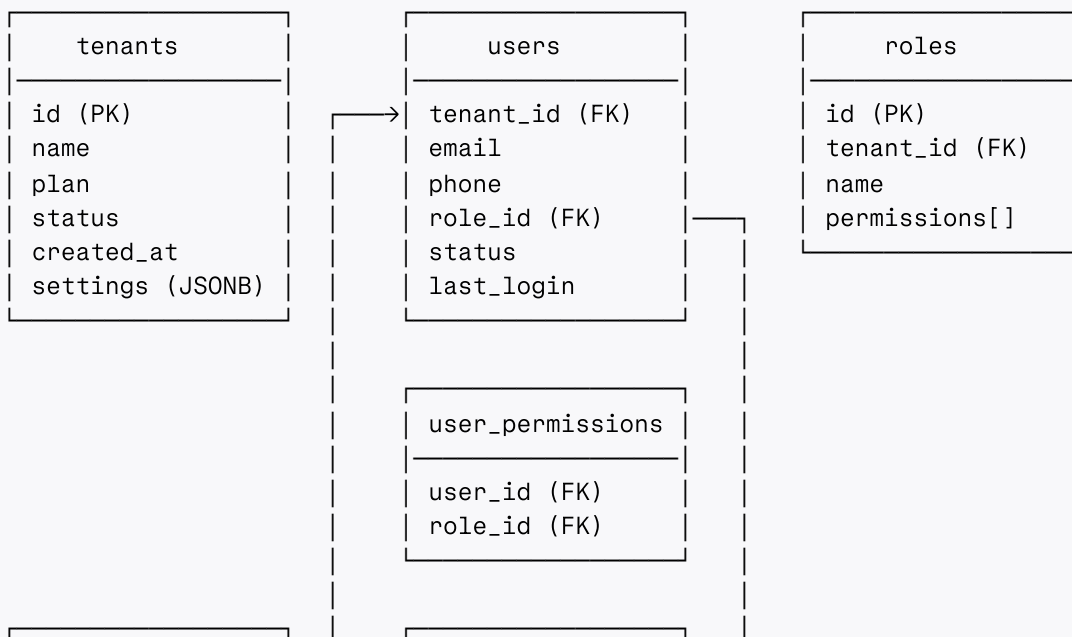
Request/Response Envelope

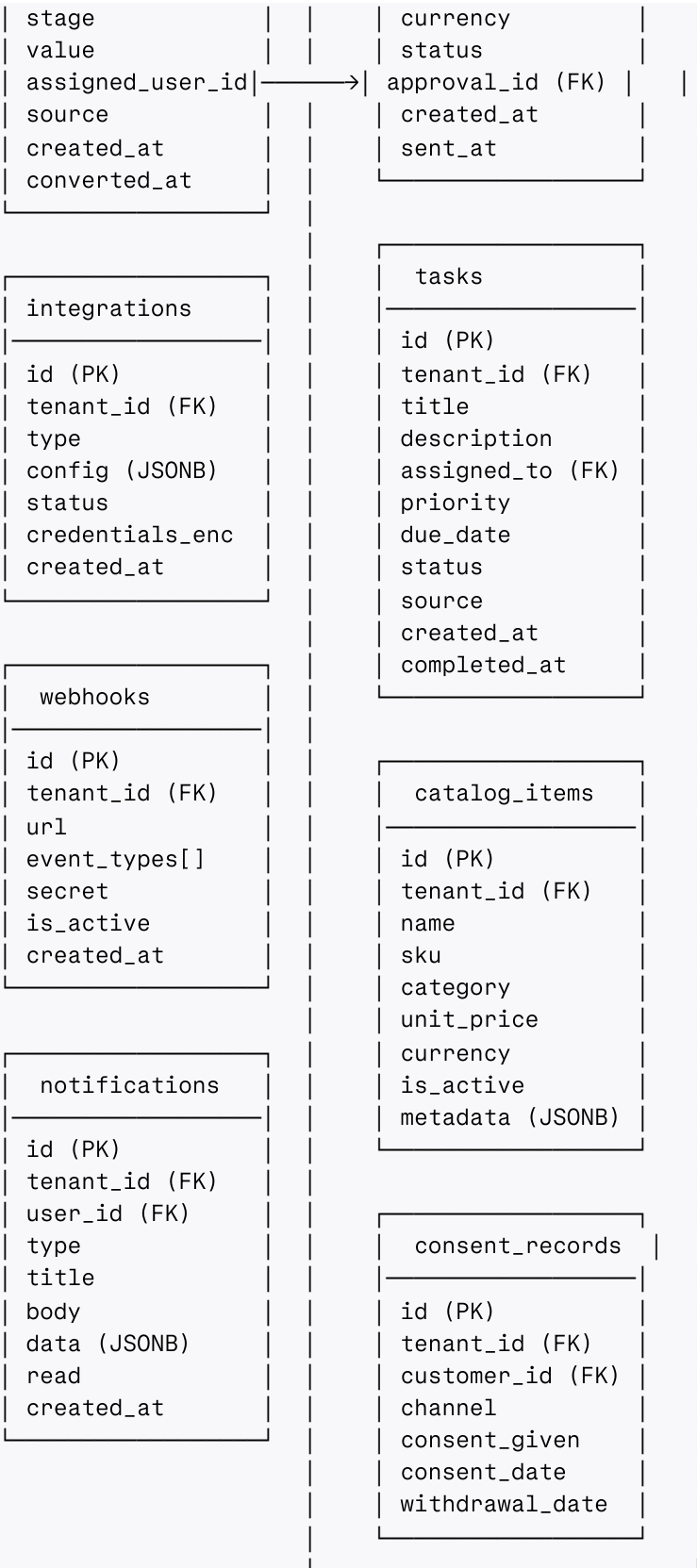
```
{
  "success": true,
  "data": { ... },
  "meta": {
    "page": 1,
    "per_page": 50,
    "total": 1250,
    "has_more": true
  }
}
```

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Phone number format is invalid",
    "details": [
      { "field": "phone", "issue": "Must include country code" }
    ]
  }
}
```

4. Database Schema

Entity Relationship Overview





Detailed Table Definitions (SQL DDL)

```
-- =====
-- TENANTS
-- =====

CREATE TABLE tenants (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(200) NOT NULL,
  slug        VARCHAR(100) UNIQUE NOT NULL,
  plan        VARCHAR(20) DEFAULT 'starter', -- starter, growth, enterprise
  status      VARCHAR(20) DEFAULT 'trial',   -- trial, active, suspended, cancelled
  trial_ends_at TIMESTAMPTZ,
  settings    JSONB DEFAULT '{}'::jsonb,
  -- settings includes: default_language, currency, timezone,
  -- whatsapp_phone_id, approval_thresholds, retention_days
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- USERS
-- =====

CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  email       VARCHAR(255) NOT NULL,
  phone       VARCHAR(30),
  full_name   VARCHAR(200) NOT NULL,
  password_hash VARCHAR(255),
  role        VARCHAR(50) DEFAULT 'staff', -- owner, admin, manager, staff
  status      VARCHAR(20) DEFAULT 'invited', -- invited, active, disabled
  preferences JSONB DEFAULT '{}'::jsonb,
  -- preferences: language, notification_channels, dashboard_layout
  last_login_at TIMESTAMPTZ,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(tenant_id, email)
);

-- =====
-- CUSTOMERS
-- =====

CREATE TABLE customers (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  name        VARCHAR(200) NOT NULL,
  phone       VARCHAR(30),
  email       VARCHAR(255),
  company     VARCHAR(200),
  customer_type VARCHAR(50), -- individual, business
  status      VARCHAR(30) DEFAULT 'active', -- active, inactive, blacklisted
  metadata    JSONB DEFAULT '{}'::jsonb,
  -- metadata: industry, tags, custom_fields
  consent_status VARCHAR(20) DEFAULT 'pending', -- pending, granted, withdrawn
  consent_date  TIMESTAMPTZ,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
```

```

        updated_at      TIMESTAMPTZ DEFAULT NOW()
    );

-- =====
-- CONVERSATIONS
-- =====
CREATE TABLE conversations (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id           UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    customer_id         UUID REFERENCES customers(id) ON DELETE SET NULL,
    channel              VARCHAR(30) NOT NULL, -- whatsapp, email, web_chat, social
    external_id         VARCHAR(255), -- WhatsApp conversation ID, email thread ID, etc
    status              VARCHAR(30) DEFAULT 'open', -- open, pending_approval, closed, etc
    assigned_user_id    UUID REFERENCES users(id),
    priority             VARCHAR(20) DEFAULT 'normal', -- low, normal, high, urgent
    intent              VARCHAR(100), -- inquiry, complaint, booking, support, etc.
    context              JSONB DEFAULT '{}':jsonb,
    -- context: conversation_summary, customer_history_summary, ai_context
    ai_draft            JSONB, -- pending AI-drafted response
    last_message_at     TIMESTAMPTZ,
    created_at          TIMESTAMPTZ DEFAULT NOW(),
    updated_at          TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- MESSAGES
-- =====
CREATE TABLE messages (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    conversation_id     UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    tenant_id           UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    sender_type         VARCHAR(20) NOT NULL, -- customer, staff, ai, system
    sender_id           UUID, -- user_id or AI agent ID
    content             TEXT,
    content_type        VARCHAR(30) DEFAULT 'text', -- text, image, document, audio, etc
    media_url           VARCHAR(500),
    language            VARCHAR(10) DEFAULT 'ar', -- ar, en
    ai_generated        BOOLEAN DEFAULT FALSE,
    ai_metadata         JSONB, -- model, prompt_version, confidence, tokens_used
    approval_id         UUID, -- FK to approvals if AI-generated and pending
    status              VARCHAR(30) DEFAULT 'sent', -- draft, pending_approval, approved
    external_msg_id     VARCHAR(255), -- WhatsApp message ID, email message ID
    created_at          TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- LEADS
-- =====
CREATE TABLE leads (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id           UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    customer_id         UUID REFERENCES customers(id),
    conversation_id     UUID REFERENCES conversations(id),
    stage              VARCHAR(50) DEFAULT 'new', -- new, qualified, quoted, won, lost
    value              DECIMAL(12, 2),
    currency            VARCHAR(3) DEFAULT 'QAR',

```

```

    assigned_user_id UUID REFERENCES users(id),
    source            VARCHAR(50), -- whatsapp, web_chat, email, manual, social
    notes             TEXT,
    metadata          JSONB DEFAULT '{}'::jsonb,
    created_at        TIMESTAMPTZ DEFAULT NOW(),
    updated_at        TIMESTAMPTZ DEFAULT NOW(),
    converted_at       TIMESTAMPTZ,
    converted_to_customer_id UUID REFERENCES customers(id)
);

-- =====
-- QUOTATIONS
-- =====
CREATE TABLE quotations (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id         UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    customer_id       UUID REFERENCES customers(id),
    lead_id           UUID REFERENCES leads(id),
    quotation_number  VARCHAR(50),
    items             JSONB NOT NULL,
    -- items: [{ product_id, name, qty, unit_price, discount, total }]
    subtotal          DECIMAL(12, 2),
    discount          DECIMAL(12, 2) DEFAULT 0,
    tax               DECIMAL(12, 2) DEFAULT 0,
    total             DECIMAL(12, 2),
    currency          VARCHAR(3) DEFAULT 'QAR',
    status            VARCHAR(30) DEFAULT 'draft', -- draft, pending_approval, approved
    approval_id       UUID,
    notes             TEXT,
    valid_until       DATE,
    created_by        UUID REFERENCES users(id),
    ai_generated       BOOLEAN DEFAULT FALSE,
    created_at        TIMESTAMPTZ DEFAULT NOW(),
    sent_at           TIMESTAMPTZ,
    updated_at        TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- DOCUMENTS
-- =====
CREATE TABLE documents (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id         UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    uploaded_by       UUID REFERENCES users(id),
    doc_type          VARCHAR(50), -- invoice, contract, delivery_note, purchase_order
    file_name         VARCHAR(255),
    file_url          VARCHAR(500) NOT NULL, -- S3/MinIO presigned or path
    file_size         BIGINT,
    mime_type         VARCHAR(100),
    extracted_data     JSONB, -- structured extraction result
    extraction_status VARCHAR(30) DEFAULT 'pending', -- pending, processing, completed
    raw_text          TEXT, -- OCR raw output
    metadata          JSONB DEFAULT '{}'::jsonb,
    created_at        TIMESTAMPTZ DEFAULT NOW(),
    updated_at        TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- =====
-- TASKS (ACTION ITEMS)
-- =====

CREATE TABLE tasks (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id         UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    title             VARCHAR(300) NOT NULL,
    description        TEXT,
    assigned_to        UUID REFERENCES users(id),
    priority           VARCHAR(20) DEFAULT 'normal', -- low, normal, high, urgent
    status             VARCHAR(30) DEFAULT 'pending', -- pending, in_progress, complete
    source             VARCHAR(50), -- ai, manual, workflow, system
    source_id          UUID, -- conversation_id, lead_id, etc.
    due_date           TIMESTAMPTZ,
    completed_at       TIMESTAMPTZ,
    completed_by       UUID REFERENCES users(id),
    created_at         TIMESTAMPTZ DEFAULT NOW(),
    updated_at         TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- APPROVALS
-- =====

CREATE TABLE approvals (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id         UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    conversation_id    UUID REFERENCES conversations(id),
    entity_type        VARCHAR(50) NOT NULL, -- message, quotation, document, workflow
    entity_id          UUID NOT NULL,
    requested_by       VARCHAR(20) NOT NULL, -- ai, system
    approved_by        UUID REFERENCES users(id),
    status             VARCHAR(30) DEFAULT 'pending', -- pending, approved, rejected, etc.
    content            JSONB, -- snapshot of what's being approved
    reason             TEXT, -- rejection reason or approval note
    created_at         TIMESTAMPTZ DEFAULT NOW(),
    resolved_at        TIMESTAMPTZ
);

-- =====
-- AUDIT LOG
-- =====

CREATE TABLE audit_log (
    id                BIGSERIAL PRIMARY KEY,
    tenant_id         UUID NOT NULL,
    user_id           UUID,
    actor_type         VARCHAR(20), -- user, ai, system
    action             VARCHAR(100) NOT NULL, -- create, update, delete, approve, reject
    entity_type        VARCHAR(50), -- customer, conversation, quotation, etc.
    entity_id          UUID,
    before_data        JSONB,
    after_data         JSONB,
    ip_address         INET,
    user_agent         TEXT,
    metadata           JSONB DEFAULT '{}'::jsonb,
    created_at         TIMESTAMPTZ DEFAULT NOW()
);

```

```

);
-- Partitioned by month for performance
CREATE INDEX idx_audit_tenant_created ON audit_log(tenant_id, created_at DESC);

-- =====
-- INTEGRATIONS
-- =====
CREATE TABLE integrations (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id         UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    type              VARCHAR(50) NOT NULL, -- whatsapp, email, calendar, crm, account
    config            JSONB NOT NULL,
    -- whatsapp: { phone_number_id, waba_id, verified_name }
    -- email: { smtp_host, smtp_port, imap_host, imap_port, email_address }
    credentials_enc    BYTEA, -- AES-256 encrypted credentials
    status            VARCHAR(20) DEFAULT 'disconnected', -- connected, disconnected,
    last_synced_at    TIMESTAMPTZ,
    created_at        TIMESTAMPTZ DEFAULT NOW(),
    updated_at        TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE(tenant_id, type)
);

-- =====
-- WEBHOOKS (Outbound)
-- =====
CREATE TABLE webhooks (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id         UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    url              VARCHAR(500) NOT NULL,
    event_types       TEXT[] NOT NULL, -- conversation.created, lead.converted, quota
    secret            VARCHAR(255), -- HMAC signing secret
    is_active         BOOLEAN DEFAULT TRUE,
    last_triggered     TIMESTAMPTZ,
    failure_count     INTEGER DEFAULT 0,
    created_at        TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- NOTIFICATIONS
-- =====
CREATE TABLE notifications (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id         UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    user_id           UUID REFERENCES users(id),
    type              VARCHAR(50), -- new_message, approval_required, task_due, lead
    title             VARCHAR(300),
    body              TEXT,
    data              JSONB DEFAULT '{}'::jsonb,
    read_at           TIMESTAMPTZ,
    created_at        TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- CONSENT RECORDS (PDPL Compliance)
-- =====
CREATE TABLE consent_records (

```

```

    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id     UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    customer_id   UUID REFERENCES customers(id),
    channel       VARCHAR(30), -- whatsapp, email, sms, all
    consent_type  VARCHAR(50), -- marketing, service, data_processing
    consent_given BOOLEAN NOT NULL,
    consent_date  TIMESTAMPTZ,
    withdrawal_date TIMESTAMPTZ,
    ip_address    INET,
    created_at    TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- CATALOG ITEMS (Product/Service Catalog)
-- =====

CREATE TABLE catalog_items (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id     UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    name          VARCHAR(300) NOT NULL,
    name_ar       VARCHAR(300), -- Arabic name
    sku           VARCHAR(100),
    category       VARCHAR(100),
    description    TEXT,
    unit_price     DECIMAL(12, 2) NOT NULL,
    currency       VARCHAR(3) DEFAULT 'QAR',
    unit           VARCHAR(30) DEFAULT 'piece', -- piece, hour, day, service
    is_active      BOOLEAN DEFAULT TRUE,
    metadata       JSONB DEFAULT '{}'::jsonb,
    created_at    TIMESTAMPTZ DEFAULT NOW(),
    updated_at    TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- WORKFLOW TEMPLATES
-- =====

CREATE TABLE workflow_templates (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id     UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    name          VARCHAR(200) NOT NULL,
    description    TEXT,
    trigger        JSONB NOT NULL,
    -- trigger: { type: "incoming_message", conditions: { intent: "inquiry", channel
    steps          JSONB NOT NULL,
    -- steps: [{ type: "ai_draft", config: {...} }, { type: "approval", config: {...}
    is_active      BOOLEAN DEFAULT TRUE,
    created_at    TIMESTAMPTZ DEFAULT NOW(),
    updated_at    TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE workflow_runs (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id     UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    template_id   UUID REFERENCES workflow_templates(id),
    trigger_entity VARCHAR(50),
    trigger_entity_id UUID,
    status        VARCHAR(30) DEFAULT 'running', -- running, completed, failed, ca

```

```

    current_step    INTEGER DEFAULT 0,
    context         JSONB DEFAULT '{} '::jsonb,
    started_at      TIMESTAMPTZ DEFAULT NOW(),
    completed_at    TIMESTAMPTZ
);

-- =====
-- ROW-LEVEL SECURITY POLICIES
-- =====

ALTER TABLE users ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_users ON users
    USING (tenant_id = current_setting('app.tenant_id')::UUID);

ALTER TABLE customers ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_customers ON customers
    USING (tenant_id = current_setting('app.tenant_id')::UUID);

ALTER TABLE conversations ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_conversations ON conversations
    USING (tenant_id = current_setting('app.tenant_id')::UUID);

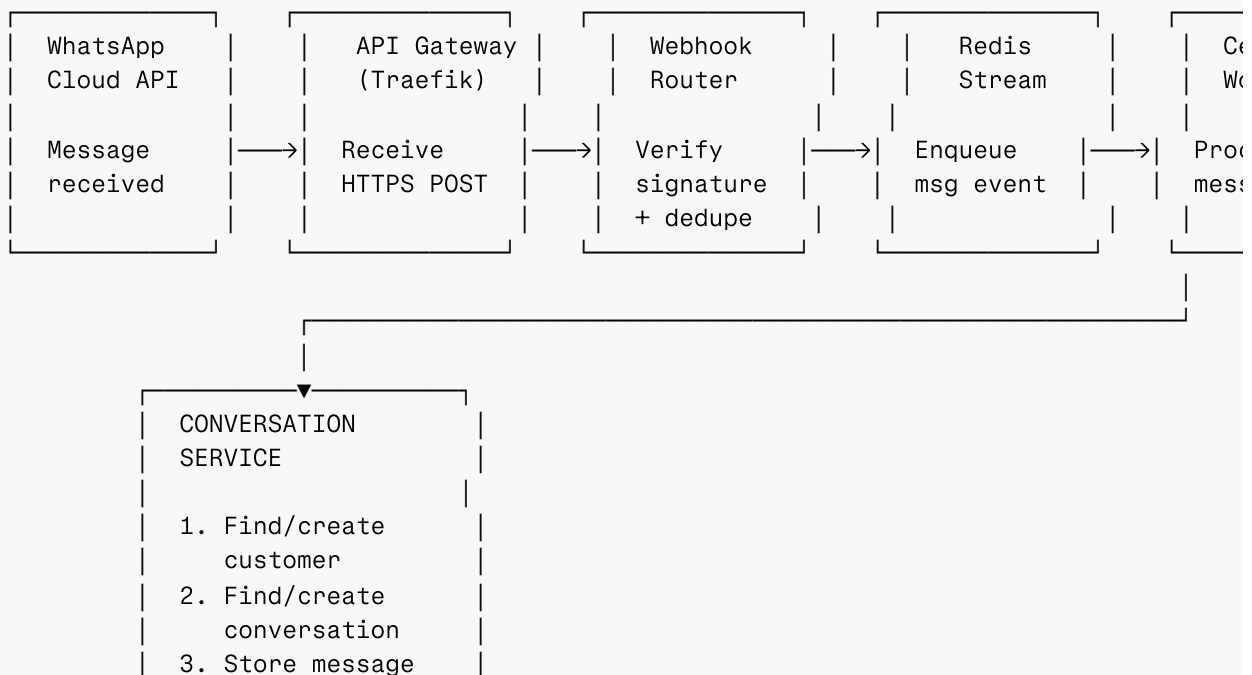
ALTER TABLE messages ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_messages ON messages
    USING (tenant_id = current_setting('app.tenant_id')::UUID);

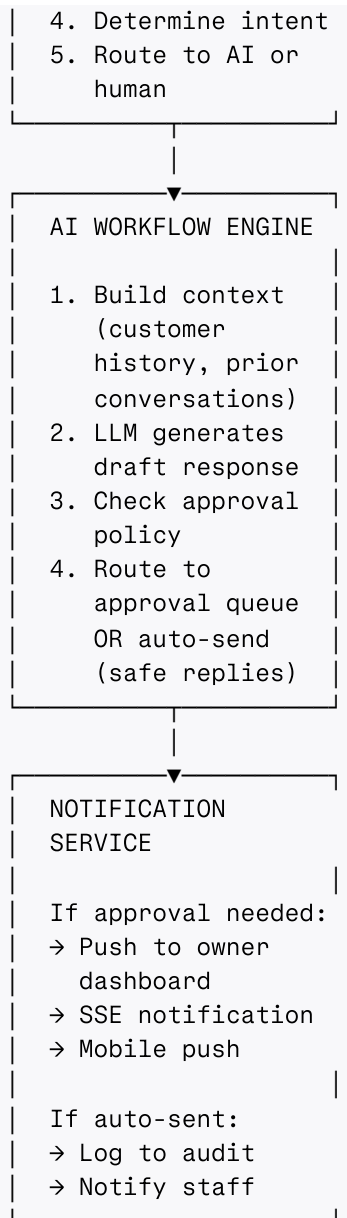
-- ... (repeat for all tenant-scoped tables)
-- Application sets app.tenant_id before each query:
-- SET LOCAL app.tenant_id = '<uuid>';

```

5. Webhook Integration Flow

Inbound Webhook Flow (WhatsApp → Namaa)





WhatsApp Webhook Verification Flow

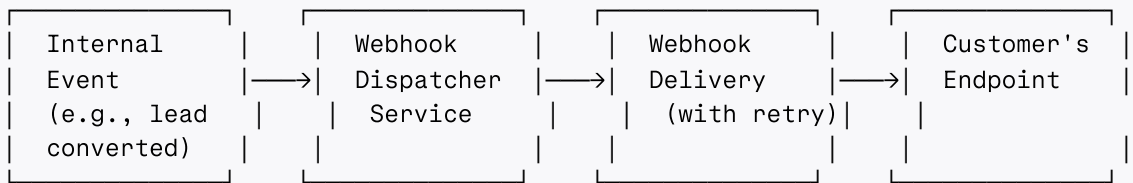
- Step 1: Meta sends GET request with hub.challenge for verification
GET /api/v1/webhooks/whatsapp?hub.mode=subscribe&hub.challenge=XXXX&hub.verify_token=XXXX
- Step 2: Namaa verifies hub.verify_token matches tenant's configured token
→ Returns hub.challenge as plain text (HTTP 200)
- Step 3: Meta sends POST with message webhook
POST /api/v1/webhooks/whatsapp
X-Hub-Signature-256: sha256=XXXXX (HMAC-SHA256 of payload)
- Step 4: Namaa verifies signature:
expected = HMAC_SHA256(payload, app_secret)
if signature != expected → reject (HTTP 403)
- Step 5: Namaa responds HTTP 200 immediately (within 5 seconds)
→ Message is enqueued for async processing

→ Meta retries if no 200 within timeout

Step 6: Celery worker processes the message:

- a. Parse message type (text, image, document, audio, location)
- b. Extract sender phone, message content, timestamp
- c. Identify/create customer record
- d. Find or create conversation
- e. Store message in DB
- f. Trigger AI workflow engine
- g. Generate draft response (or auto-respond for FAQ)
- h. If approval required → create approval record + notify
- i. If auto-approved → send via WhatsApp API + log to audit

Outbound Webhook Flow (Namaa → Customer Systems)



Webhook event types:

Event	Trigger	Payload
conversation.created	New conversation started	conversation_id, customer_id, channel
conversation.assigned	Conversation assigned to staff	conversation_id, assigned_user_id
conversation.closed	Conversation marked closed	conversation_id, resolution_summary
lead.created	New lead created	lead_id, customer_id, source, value
lead.stage_changed	Lead moved to new stage	lead_id, old_stage, new_stage
lead.converted	Lead converted to customer	lead_id, customer_id
quotation.created	New quotation generated	quotation_id, customer_id, total
quotation.sent	Quotation sent to customer	quotation_id, channel, sent_at
quotation.approved	Quotation approved internally	quotation_id, approved_by
message.ai_generated	AI drafted a message	conversation_id, message_id, draft
message.approved	AI message approved by human	conversation_id, message_id, approved_by
message.sent	Message sent to customer	conversation_id, channel, message_id
document.processed	Document extraction completed	document_id, doc_type, extracted_data
task.created	New task created	task_id, title, assigned_to, due_date
task.completed	Task marked complete	task_id, completed_by
approval.required	Approval needed	approval_id, entity_type, entity_id

Event	Trigger	Payload
approval.resolved	Approval approved/rejected	approval_id, status, resolved_by

Webhook delivery guarantees:

- HMAC-SHA256 signature in X-Namaa-Signature header
- Exponential backoff retry: 1s, 5s, 30s, 2m, 15m, 1h, 6h, 24h (max 8 attempts)
- Dead letter queue for permanently failed webhooks
- Webhook delivery log with status, response code, and attempt count

Email Integration Flow

Option A: IMAP Polling (Phase 1 – simpler)

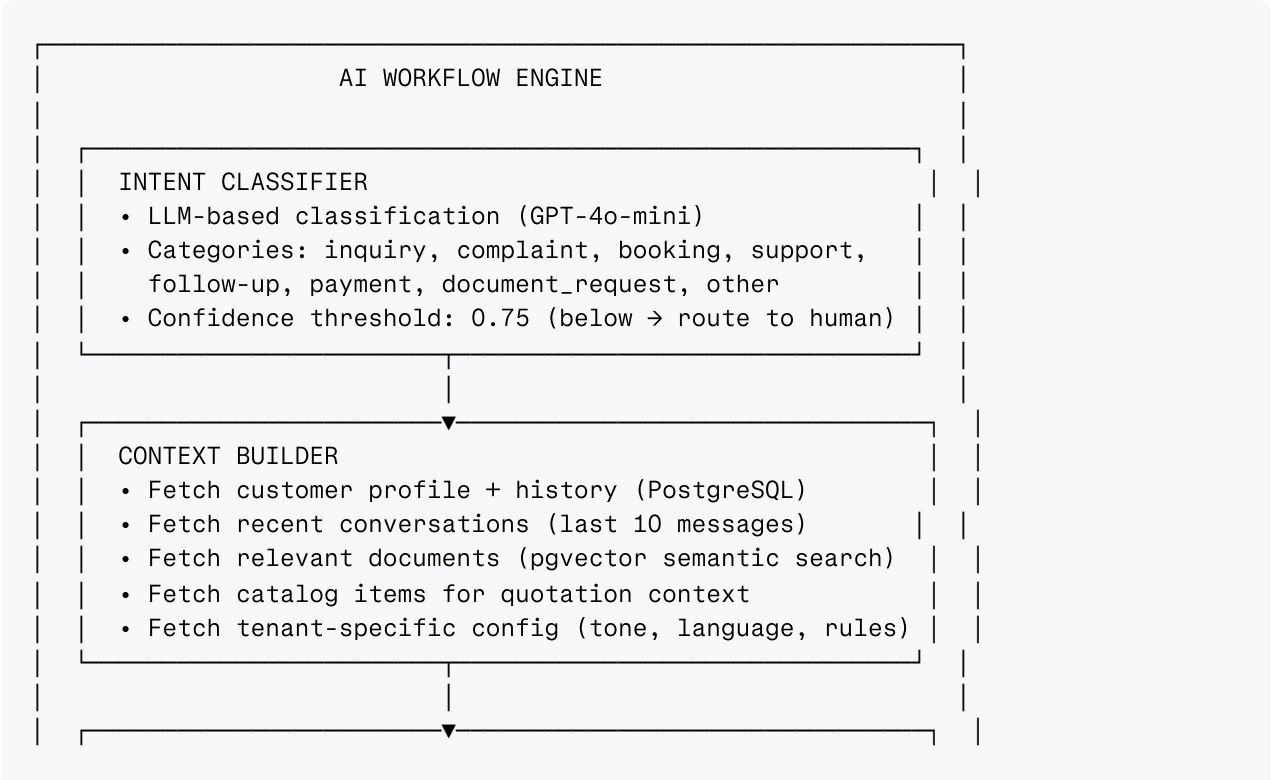
Celery Beat (every 60s) → IMAP fetch new emails →
Parse (sender, subject, body) → Create conversation →
Trigger AI workflow → Draft reply → Approval queue

Option B: Email Webhook / Push (Phase 2 – richer)

Microsoft Graph / Gmail API push notification →
Webhook Router → Fetch full email → Process

6. AI & Workflow Engine Design

AI Orchestration Architecture



RESPONSE GENERATOR

- System prompt: tenant-specific persona + rules
- Language detection: Arabic or English
- Function calling: create_lead, create_quotation, create_task, schedule_followup, escalate_to_human
- Output: draft message + proposed actions

APPROVAL POLICY ENGINE

- Rule-based: what requires human approval?
 - All outbound messages (Phase 1: always)
 - Quotations with discount > threshold
 - Messages to VIP customers
 - Any document sharing
 - Payment-related actions
- Auto-send rules (Phase 2+):
 - FAQ responses (confidence > 0.9)
 - Acknowledgments ("we received your message")
 - Status updates ("your order is ready")

ACTION EXECUTOR

- Send message via WhatsApp API / Email / SMS
- Create lead in pipeline
- Generate quotation from catalog
- Create task for staff
- Schedule follow-up reminder
- Trigger outbound webhook
- Log every action to audit trail

LLM Prompt Architecture

SYSTEM PROMPT (Tenant-specific):

You are {tenant_name}'s AI operations assistant.
Your role: Help manage customer communications and operations.
Language: Respond in {detected_language} (Arabic or English).
Tone: Professional, helpful, concise.
Rules:

- Never make commitments without approval
- Never share pricing unless from the approved catalog
- Always offer to escalate to a human for complex issues
- Include the customer's name in responses
- Keep responses under 150 words unless asked for detail

Available actions (function calls):

- create_lead(customer_id, source, value, notes)
- create_quotation(customer_id, items[], notes)
- create_task(title, assigned_to, priority, due_date)
- schedule_followup(customer_id, delay_hours, message_draft)
- escalate_to_human(conversation_id, reason, priority)

- send_acknowledgment(message_template)

USER CONTEXT (Dynamic):

Customer: {name}, {company}, {phone}

Previous interactions: {summary}

Customer tags: {tags}

Recent conversation: {last_10_messages}

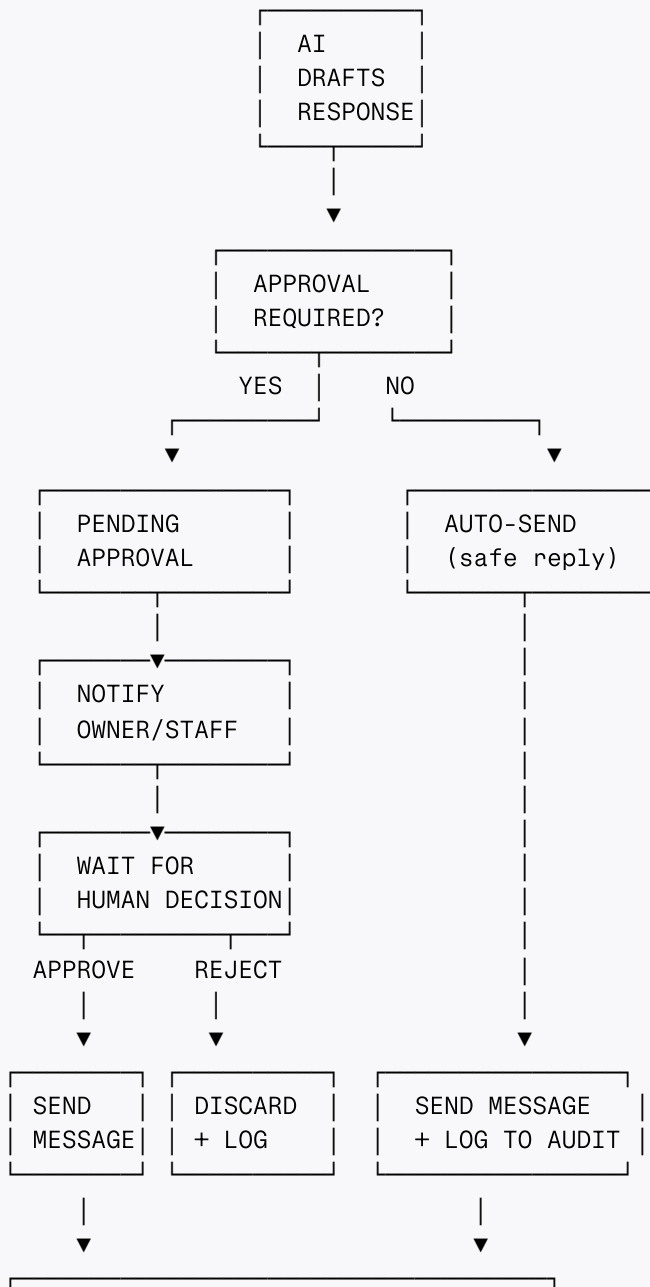
Relevant documents: {semantic_search_results}

Catalog items matching inquiry: {matched_products}

INCOMING MESSAGE:

{customer_message}

Approval Workflow State Machine



```
LOG TO AUDIT TRAIL
• Who approved/auto-sent
• What was sent
• AI confidence score
• Model used
• Response time
```

7. Security & Compliance Architecture

Security Layers

SECURITY ARCHITECTURE

LAYER 1: NETWORK SECURITY

- TLS 1.3 everywhere (Traefik auto-cert)
- WAF rules (OWASP Top 10 protection)
- DDoS protection via Cloudflare or AWS Shield
- API Gateway rate limiting (per-tenant, per-IP)
- IP allowlist for admin endpoints

LAYER 2: AUTHENTICATION

- JWT access tokens (15-min expiry)
- JWT refresh tokens (7-day expiry, rotating)
- OTP via phone/email for sensitive actions
- OAuth 2.0 for social login (future)

LAYER 3: AUTHORIZATION (RBAC)

- Roles: owner, admin, manager, staff, viewer
- Per-endpoint permission checks
- Tenant-scoped access (RLS in PostgreSQL)
- Approval thresholds per role

LAYER 4: DATA PROTECTION

- AES-256 encryption for credentials at rest
- Field-level encryption for PII (phone, email)
- Data masking in logs (phone, email, card numbers)
- Secure credential storage (Vault/AWS Secrets Mgr)
- Encrypted backups

LAYER 5: AUDIT & MONITORING

- Immutable audit log (append-only)
- Every data access logged

- AI action traceability (model, prompt, output)
- Anomaly detection (unusual access patterns)
- Real-time alerting (failed logins, mass exports)

Qatar PDPL Compliance Checklist

Requirement	Implementation
Lawful basis for processing	Consent management system; consent records table
Data subject rights	API endpoints for access, rectification, erasure, portability
Data processing records	Audit log captures all processing activities
Privacy by design	RLS, field encryption, data minimization in AI prompts
Data retention	Configurable retention policies per data type; auto-purge
Third-party processor management	Integration registry with DPA tracking
Breach notification	Alerting pipeline for security incidents
Data localization option	Self-hosted deployment option in Qatar (Mada or Ooredoo cloud)
Cross-border transfer controls	Configurable data residency settings per tenant

Data Retention Policy

```
# Retention policy configuration (per tenant, configurable)
RETENTION_POLICIES = {
    "conversations": {
        "active_retention": "2 years",      # Keep while customer is active
        "archive_after": "2 years",        # Move to cold storage
        "delete_after": "5 years",         # Permanent deletion
    },
    "messages": {
        "delete_after": "3 years",         # Auto-delete old messages
    },
    "documents": {
        "delete_after": "5 years",         # Documents kept longer
    },
    "audit_log": {
        "delete_after": "7 years",         # Compliance requirement
    },
    "ai_metadata": {
        "delete_after": "1 year",         # AI prompts/outputs
    },
}
# Celery Beat runs daily to enforce retention policies
```

8. Phased Deployment Roadmap

Phase 0: Foundation (Weeks 1-3)

Goal: Stand up core infrastructure and development environment.

Week	Deliverable	Details
1	Project scaffold	Monorepo structure, Docker Compose dev environment, CI/CD pipeline
1	Database setup	PostgreSQL 16 + Redis + MinIO running; initial migrations applied
2	Auth system	JWT auth, OTP, user registration, tenant creation, RBAC roles
2	API skeleton	FastAPI project structure, OpenAPI docs, health checks, error handling
3	Frontend scaffold	Next.js project, Tailwind, RTL/LTR i18n, login/dashboard shell
3	Multi-tenancy	RLS policies, tenant context middleware, tenant-scoped queries

Exit criteria: A developer can create a tenant, register a user, log in, and see an empty dashboard.

Phase 1: WhatsApp + AI Core (Weeks 4-8)

Goal: Receive WhatsApp messages, generate AI draft responses, and require human approval.

Week	Deliverable	Details
4	WhatsApp integration	Meta Business API setup, webhook verification, message receiving
4	Conversation engine	Customer matching/creation, conversation lifecycle, message storage
5	AI intent classification	LLM-based intent detection, confidence scoring, routing logic
5	Context builder	Customer history retrieval, conversation context window management
6	AI response generation	LLM prompt architecture, bilingual response generation, function calling
6	Approval workflow	Draft → approval queue → owner notification → approve/reject → send
7	WhatsApp outbound	Send approved messages via WhatsApp Cloud API, template management
7	Audit logging	All actions logged: AI drafts, approvals, sends, customer interactions
8	Owner dashboard	"Today's actions" view, pending approvals, conversation list
8	Real-time notifications	SSE for live updates, push notifications via FCM

Exit criteria: A business owner receives a WhatsApp message, the AI drafts a bilingual reply, the owner approves it from the dashboard, and it's sent back — all within 2 minutes.

Phase 2: Lead Pipeline + Quotations (Weeks 9-13)

Goal: Convert conversations into leads and generate quotations from a product catalog.

Week	Deliverable	Details
9	Lead pipeline	Lead creation from conversations, stage management, assignment
9	Lead dashboard	Pipeline view (kanban), lead stats, conversion metrics
10	Product catalog	CRUD for catalog items, bilingual names, categories, pricing
10	Quotation engine	AI-assisted quotation generation from catalog, manual editing
11	Quotation approval	Discount threshold approval, multi-level approval for large quotes
11	Quotation sending	Send quotations via WhatsApp (PDF/image), email
12	Follow-up automation	AI drafts follow-up messages for cold leads, scheduled reminders
12	Task management	Action items created from conversations, assigned to staff, due dates
13	Dashboard metrics	Lead response time, quote-to-sale conversion, staff activity

Exit criteria: A WhatsApp inquiry becomes a lead, generates a quotation from the catalog, gets approved, and is sent — all tracked with ROI metrics.

Phase 3: Document Intelligence (Weeks 14-17)

Goal: Extract data from invoices, contracts, and forms; route for processing.

Week	Deliverable	Details
14	File upload	Document upload via dashboard and WhatsApp, S3 storage, virus scan
14	OCR integration	Google Document AI / Tesseract, text extraction, Arabic support
15	Data extraction	Structured extraction: invoice fields, contract clauses, form data
15	Extraction review	Human review interface for extracted data, correction capability
16	Document routing	Auto-create tasks from documents (e.g., "invoice received → approval task")
16	Document search	Full-text search across documents (PostgreSQL FTS → Elasticsearch later)
17	Document templates	Pre-defined extraction templates per document type

Exit criteria: A customer sends an invoice photo via WhatsApp, Namaa extracts the key fields, creates an approval task, and the owner reviews and acts on it.

Phase 4: Email + Calendar Integration (Weeks 18-21)

Goal: Extend beyond WhatsApp to email and calendar.

Week	Deliverable	Details
18	Email inbound	IMAP polling, email parsing, conversation creation from emails
18	Email outbound	SMTP sending, HTML email templates, bilingual email support
19	Email AI workflow	Same AI pipeline applied to email inquiries, approval flow
19	Calendar integration	Google Calendar / Microsoft Graph, appointment scheduling
20	Booking workflow	AI-assisted appointment booking, reminders, rescheduling
20	Reminder automation	Scheduled reminders via WhatsApp/email for appointments, payments
21	Unified inbox	Combined view of WhatsApp + email conversations in single dashboard

Exit criteria: A customer emails an inquiry, the AI drafts a bilingual reply and proposes a meeting time, the owner approves, and a calendar invite is sent.

Phase 5: Governance, Analytics & Polish (Weeks 22-26)

Goal: Production-grade compliance, analytics, and UX polish.

Week	Deliverable	Details
22	Consent management	PDPL consent capture, withdrawal, consent status tracking
22	Data retention	Automated retention enforcement, data deletion workflows
23	Audit dashboard	Audit trail viewer, export (CSV/PDF), filter by user/action/entity
23	Role management	Custom roles, permission matrix, user invitation workflow
24	ROI analytics	Hours saved, leads recovered, response time improvement, revenue impact
24	Performance metrics	API response times, AI latency, error rates, uptime dashboard
25	Onboarding wizard	Step-by-step setup for new tenants: WhatsApp connect, catalog import, team invite
25	Mobile PWA	Responsive mobile experience, push notifications, offline capability
26	Security hardening	Penetration testing, OWASP audit, rate limiting, input sanitization
26	Load testing	Stress test with 100 concurrent tenants, optimize bottlenecks

Exit criteria: The platform is production-ready, compliant with Qatar PDPL, and handles 100+ tenants with measurable ROI dashboards.

Phase 6: Scale & Expand (Weeks 27-36)

Goal: Prepare for GCC expansion and scale.

Week	Deliverable	Details
27-28	Kubernetes migration	Container orchestration, auto-scaling, health checks
28-29	Multi-region	Deploy in Qatar (primary) + UAE (secondary) for GCC latency
29-30	Vertical templates	Pre-built workflow packs: Retail, Logistics, Real Estate, Clinics
30-31	CRM integrations	HubSpot, Zoho, Salesforce connector (sync leads/customers)
31-32	Accounting integrations	QuickBooks, Xero, local accounting software (export invoices)
32-33	Advanced AI	Auto-send for safe replies (FAQ, acknowledgments, status updates)
33-34	Analytics platform	Elasticsearch deployment, advanced search, conversation analytics
34-35	White-label	Custom branding, custom domain per tenant
35-36	Public API	Developer documentation, SDK (Python/JS), webhook management UI

Exit criteria: Namaa operates across Qatar and UAE with vertical templates, integrations, and a public API.

9. Milestones & Deliverables

Milestone Summary

Phase 0: Foundation		Week 3	✓	Dev-ready
Phase 1: WhatsApp + AI Core		Week 8	✓	MVP
Phase 2: Leads + Quotations		Week 13	✓	Beta
Phase 3: Document Intelligence		Week 17	✓	Feature-co
Phase 4: Email + Calendar		Week 21	✓	Multi-chan
Phase 5: Governance + Polish		Week 26	✓	Production
Phase 6: Scale + Expand		Week 36	✓	GCC-ready

Key Milestone Gates

Milestone	Target Date	Criteria	Business Value
M0: Dev-Ready	Week 3	Infra running, auth working, dashboard shell	Development velocity
M1: MVP	Week 8	WhatsApp → AI draft → approval → send	First demo to design partners
M2: Beta	Week 13	Leads, quotations, follow-ups, ROI metrics	Onboard 5 paying design partners
M3: Feature-Complete	Week 17	Document intelligence, OCR, routing	Expand to document-heavy sectors

Milestone	Target Date	Criteria	Business Value
M4: Multi-Channel	Week 21	Email + calendar + unified inbox	Reduce WhatsApp-only dependency
M5: Production-Ready	Week 26	PDPL compliant, security audited, 100 tenants	Public launch in Qatar
M6: GCC-Ready	Week 36	Multi-region, vertical templates, integrations	Expand to UAE + Saudi Arabia

Team Composition by Phase

Role	Phase 0-1	Phase 2-3	Phase 4-5	Phase 6
Backend Engineer	1 (you)	2	2-3	3
Frontend Engineer	0-1	1	2	2
AI/ML Engineer	0 (use APIs)	1	1	1-2
DevOps	0 (managed)	0-1	1	1
Product/Design	0-1	1	1	1
QA	0	0-1	1	1

Budget Estimates (Directional)

Phase	Duration	Est. Cost (QAR)	Key Spend
Phase 0-1	8 weeks	50,000 - 80,000	Cloud, LLM API, WhatsApp API, dev tools
Phase 2-3	9 weeks	60,000 - 100,000	OCR API, additional dev, QA
Phase 4-5	9 weeks	70,000 - 120,000	Security audit, infrastructure, compliance
Phase 6	10 weeks	100,000 - 200,000	Multi-region, integrations, scaling
Total (Year 1)	36 weeks	280,000 - 500,000	

Costs exclude full-time salaries. Assumes founder-led development with 1-2 contractors initially.

10. Infrastructure & DevOps

Development Environment

```
# docker-compose.yml (simplified)
version: '3.8'

services:
  # — Application —
  api:
    build: ./backend
```

```
ports: ["8000:8000"]
environment:
  - DATABASE_URL=postgresql+asyncpg://namaa:namaa@db:5432/namaa
  - REDIS_URL=redis://redis:6379/0
  - S3_ENDPOINT=http://minio:9000
depends_on: [db, redis, minio]

worker:
  build: ./backend
  command: celery -A namaa.worker worker -l info -Q default,ai,notifications,webhooks
  environment:
    - DATABASE_URL=postgresql+asyncpg://namaa:namaa@db:5432/namaa
    - REDIS_URL=redis://redis:6379/0
  depends_on: [db, redis]

beat:
  build: ./backend
  command: celery -A namaa.worker beat -l info
  depends_on: [redis]

frontend:
  build: ./frontend
  ports: ["3000:3000"]
  environment:
    - NEXT_PUBLIC_API_URL=http://localhost:8000

# — Data Layer —
db:
  image: postgres:16
  environment:
    POSTGRES_DB: namaa
    POSTGRES_USER: namaa
    POSTGRES_PASSWORD: namaa
  volumes:
    - pgdata:/var/lib/postgresql/data
  ports: ["5432:5432"]

redis:
  image: redis:7-alpine
  ports: ["6379:6379"]

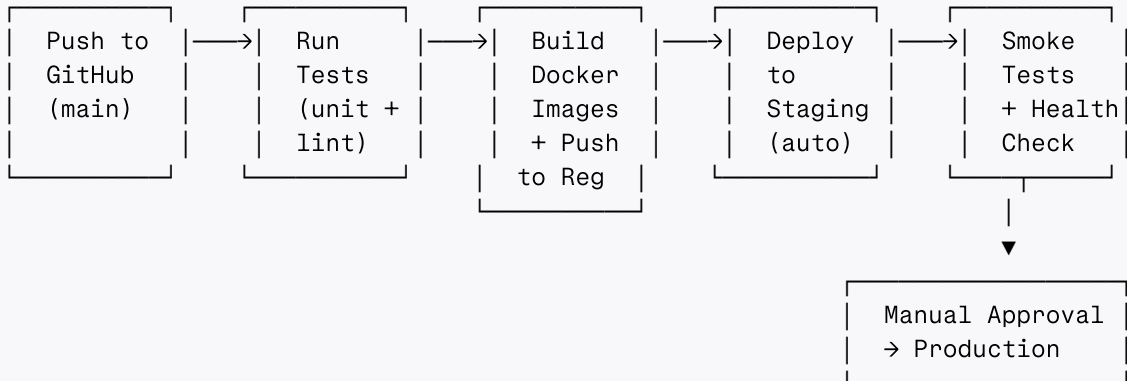
minio:
  image: minio/minio
  command: server /data --console-address ":9001"
  ports: ["9000:9000", "9001:9001"]
  environment:
    MINIO_ROOT_USER: minio
    MINIO_ROOT_PASSWORD: minio123
  volumes:
    - miniodata:/data

# — Observability —
prometheus:
  image: prom/prometheus
  ports: ["9090:9090"]
```

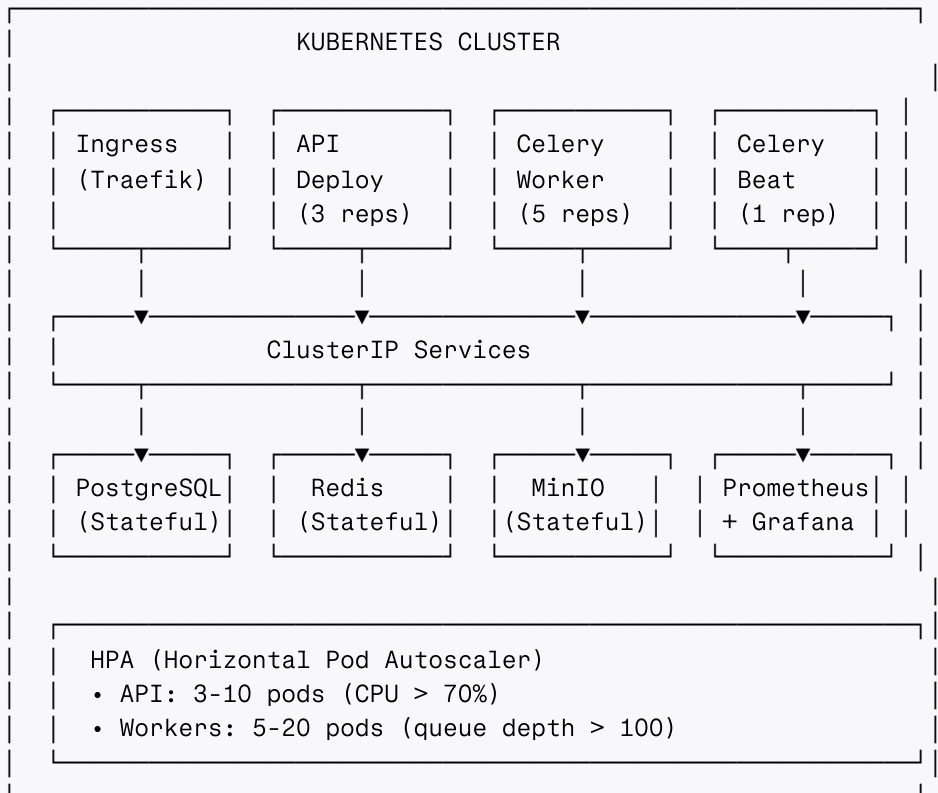
```
grafana:
  image: grafana/grafana
  ports: ["3001:3000"]
```

```
volumes:
  pgdata:
  miniodata:
```

CI/CD Pipeline



Production Deployment (Kubernetes)



Monitoring & Alerting

Metric	Alert Threshold	Action
API response time (p95)	> 2 seconds	PagerDuty: warning
API error rate	> 1%	PagerDuty: critical
Celery queue depth	> 500 messages	Auto-scale workers
WhatsApp webhook failures	> 5 in 10 min	Email + PagerDuty
LLM API errors	> 10 in 5 min	Switch to fallback provider
Database connection pool	> 80% used	Alert + investigate
Disk usage	> 85%	Alert + cleanup
Tenant error spike	Any tenant > 10 errors/hr	Alert + investigate

Appendix A: Directory Structure

```
namaa/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py                # FastAPI app entry
│   │   ├── config.py             # Settings (Pydantic BaseSettings)
│   │   ├── database.py           # Async SQLAlchemy engine + session
│   │   ├── middleware/
│   │   │   ├── tenant_context.py # Set tenant_id for RLS
│   │   │   ├── auth.py           # JWT verification
│   │   │   └── audit.py          # Auto-log mutations
│   │   ├── api/
│   │   │   ├── v1/
│   │   │   │   ├── auth.py
│   │   │   │   ├── conversations.py
│   │   │   │   ├── customers.py
│   │   │   │   ├── leads.py
│   │   │   │   ├── quotations.py
│   │   │   │   ├── documents.py
│   │   │   │   ├── tasks.py
│   │   │   │   ├── approvals.py
│   │   │   │   ├── dashboard.py
│   │   │   │   ├── integrations.py
│   │   │   │   ├── settings.py
│   │   │   │   ├── catalog.py
│   │   │   │   └── webhooks/
│   │   │   │       ├── whatsapp.py
│   │   │   │       └── email.py
│   │   │   └── deps.py           # Dependency injection
│   │   ├── models/               # SQLAlchemy models
│   │   ├── schemas/              # Pydantic schemas
│   │   ├── services/             # Business logic
│   │   │   ├── conversation_service.py
│   │   │   └── ai_engine.py
│   └── ...
```

```

├── approval_service.py
├── document_service.py
├── notification_service.py
├── whatsapp_service.py
├── email_service.py
├── webhook_service.py
├── audit_service.py
├── ai/
│   ├── prompts/          # Prompt templates
│   ├── orchestrator.py   # LLM orchestration
│   ├── intent_classifier.py
│   ├── context_builder.py
│   └── response_generator.py
├── tasks/                # Celery tasks
│   ├── process_message.py
│   ├── generate_ai_response.py
│   ├── send_notification.py
│   ├── process_document.py
│   ├── send_webhook.py
│   ├── retention_cleanup.py
│   └── email_poller.py
├── utils/
│   ├── crypto.py         # Encryption helpers
│   ├── security.py       # JWT, password hashing
│   └── logging.py        # Structured logging
├── alembic/              # Database migrations
├── tests/
├── Dockerfile
├── pyproject.toml

├── frontend/
│   ├── app/              # Next.js App Router
│   │   ├── [locale]/
│   │   │   ├── (auth)/
│   │   │   ├── (dashboard)/
│   │   │   │   ├── page.tsx      # Owner dashboard
│   │   │   │   ├── conversations/
│   │   │   │   ├── leads/
│   │   │   │   ├── quotations/
│   │   │   │   ├── documents/
│   │   │   │   ├── tasks/
│   │   │   │   ├── catalog/
│   │   │   │   ├── settings/
│   │   │   └── analytics/
│   ├── components/       # Shared UI components
│   ├── lib/              # API client, hooks, utils
│   ├── messages/         # i18n translations (ar.json, en.json)
│   ├── Dockerfile
│   └── package.json

├── infrastructure/
│   ├── docker-compose.yml # Development
│   ├── docker-compose.prod.yml # Production overrides
│   ├── k8s/              # Kubernetes manifests
│   │   ├── api-deployment.yaml
│   │   └── worker-deployment.yaml

```

```

├── postgres-statefulset.yaml
├── redis-statefulset.yaml
├── ingress.yaml
├── hpa.yaml
├── monitoring/
├── terraform/                # Infrastructure as Code (optional)
├── .github/
│   ├── workflows/
│   │   ├── ci.yaml           # Test + lint
│   │   ├── build.yaml        # Build + push images
│   │   └── deploy.yaml       # Deploy to staging/prod
└── docs/
    ├── api/                  # OpenAPI generated docs
    ├── architecture/         # ADRs (Architecture Decision Records)
    ├── runbooks/              # Operational runbooks
    └── onboarding/            # Tenant onboarding guides

```

Appendix B: Key Design Decisions

Decision	Choice	Alternatives Considered	Rationale
Architecture style	Modular monolith	Microservices, Serverless	Start simple; extract services only when scale demands it
Multi-tenancy	Shared DB + RLS	Separate DB per tenant, Schema-per-tenant	Cost-effective, strong isolation, scales to thousands of tenants
Language	Python + FastAPI	Node.js, Go, Java	Best AI ecosystem; async performance sufficient for MVP
Frontend	Next.js	Vue, Svelte, Remix	SSR + API routes + mature ecosystem + i18n support
LLM strategy	API-first (OpenAI + Anthropic)	Self-hosted (Llama, Mistral)	Faster to market; self-host for cost/sensitive data later
Queue	Celery + Redis	RabbitMQ, Kafka, SQS	Simple, Python-native, sufficient throughput for SME scale
Database	PostgreSQL (everything)	MongoDB, DynamoDB	RLS, JSONB, pgvector, FTS — one database for relational + vector + search
Deployment	Docker Compose → Kubernetes	Serverless, PaaS	Start simple; K8s for scale; avoid vendor lock-in
File storage	MinIO (S3-compatible)	Local FS, Azure Blob	Self-hosted option for Qatar data residency; S3 API compatible

This blueprint is a living document. Each phase should be validated with real customer feedback before committing to the next phase. Build narrow, prove value, then expand.